

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

§1. Sơ lược về độ phức tạp thuật toán.

1.1 Thuật toán. *Thuật toán* là một qui tắc để với những dữ liệu ban đầu đã cho, tìm được lời giải sau một khoảng thời gian hữu hạn.

1.2 Yêu cầu của thuật toán.

a) **Tính hữu hạn.** Thuật toán cần phải kết thúc sau một số hữu hạn bước. Khi thuật toán ngừng làm việc, ta phải thu được câu trả lời cho vấn đề đặt ra.

b) **Tính xác định.** Ở mỗi bước, thuật toán cần phải xác định, nghĩa là chỉ rõ việc cần làm. Nếu đối với người đọc, thuật toán chưa thoả mãn điều kiện này thì đó là lỗi của người viết!

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

c) **Tính hiệu quả.** Ngoài những yếu tố kể trên, ta còn phải xét đến tính hiệu quả của thuật toán. Có rất nhiều thuật toán, về mặt lí thuyết là kết thúc sau hữu hạn bước, tuy nhiên thời gian “hữu hạn” đó vượt quá khả năng làm việc của chúng ta. Những thuật toán đó sẽ không được xét đến.

1.3 Độ phức tạp của thuật toán.

1.3.1 Khái niệm. Thời gian làm việc của máy tính khi chạy một thuật toán nào đó không chỉ phụ thuộc vào thuật toán, mà còn phụ thuộc vào máy tính được sử dụng.

Vì thế, để có một tiêu chuẩn chung, ta sẽ đo độ phức tạp của một thuật toán bằng số các phép tính phải làm khi thực hiện thuật toán.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Khi tiến hành cùng một thuật toán, số các phép tính phải thực hiện còn phụ thuộc vào cỡ của bài toán, tức là độ lớn của đầu vào.

Vì thế, độ phức tạp của thuật toán sẽ là một hàm số của độ lớn của đầu vào. Trong những ứng dụng thực tiễn, chúng ta không cần biết chính xác hàm này, mà chỉ cần biết “cỡ” của chúng, tức là cần có một ước lượng đủ tốt để đánh giá chúng.

Một kí hiệu 0 hoặc 1 được gọi là một *bit*. Một số nguyên n biểu diễn bởi k chữ số 1 và 0 được gọi là một *số k -bit*. Độ phức tạp của một thuật toán được đo bằng số các *phép tính bit*. Phép tính bit là một phép tính logic hay số học thực hiện trên các số 1-bit 0 và 1.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

1.3.2. Định nghĩa.

Giả sử $f(n)$ và $g(n)$ là hai hàm xác định trên tập hợp các số nguyên dương. Ta nói $f(n)$ có bậc O -lớn của $g(n)$ và viết $f(n)=O(g(n))$ hoặc $f=O(g)$, nếu tồn tại một số $C > 0$ sao cho với n đủ lớn, các hàm $f(n)$ và $g(n)$ đều dương, đồng thời $f(n) < Cg(n)$.

1.3.3. Nhận xét.

1) Giả sử $f(n)$ là đa thức $f(n)=a_d n^d + a_{d-1} n^{d-1} + \dots + a_1 n + a_0$, trong đó $a_d > 0$. Ta có $f(n)=O(n^d)$.

2) Nếu $f_1(n)=O(g(n))$, $f_2(n)=O(g(n))$ thì $f_1+f_2=O(g)$.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

3) Nếu $f_1 = O(g_1)$, $f_2 = O(g_2)$, thì $f_1 \cdot f_2 = O(g_1 \cdot g_2)$.

4) Nếu tồn tại giới hạn hữu hạn $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = C$ thì $f = O(g)$.

5) Với mọi số $\varepsilon > 0$, $\ln(n) = O(n^\varepsilon)$.

1.3.4. Định nghĩa. Một thuật toán được gọi là có *độ phức tạp đa thức*, hoặc *có thời gian đa thức*, nếu số các phép tính cần thiết khi thực hiện thuật toán không vượt quá $O(\log^d n)$, trong đó n là độ lớn của đầu vào, và d là số nguyên dương nào đó.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Nói cách khác, nếu đầu vào là các số k -bit thì thời gian thực hiện thuật toán là $O(k^d)$, tức là tương đương với một đa thức của k .

Các thuật toán với thời gian $O(n^\alpha)$, $\alpha > 0$, được gọi là các thuật toán với *độ phức tạp mũ*, hoặc *thời gian mũ*.

1.3.5 Chú ý.

a) Nếu một thuật toán nào đó có độ phức tạp $O(g)$ thì cũng có thể nói nó độ phức tạp $O(h)$ với mọi hàm $h > g$. Tuy nhiên, ta luôn luôn cố gắng tìm ước lượng tốt nhất có thể được, để tránh hiểu sai về độ phức tạp thực sự của thuật toán.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

b) Có những thuật toán có độ phức tạp trung gian giữa đa thức và mũ. Ta thường gọi đó là thuật toán ***dưới mũ***. Chẳng hạn, thuật toán nhanh nhất được biết hiện nay để phân tích một số nguyên n ra thừa số là thuật toán có độ phức tạp

$$e^{\sqrt{\log n (\log \log n)}}$$

Khi giải một bài toán, không những ta chỉ cố gắng tìm ra một thuật toán nào đó, mà còn muốn tìm ra thuật toán “tốt nhất”.

Đánh giá độ phức tạp là một trong những cách để phân tích, so sánh và tìm ra thuật toán tối ưu. Tuy nhiên, độ phức tạp không phải là tiêu chuẩn duy nhất để đánh giá thuật toán. Có những thuật toán, về lý thuyết thì có độ phức tạp cao hơn một thuật toán khác, nhưng khi sử dụng lại có kết quả (gần đúng) nhanh hơn nhiều.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Chúng ta cần lưu ý thêm một điểm sau đây. Mặc dù định nghĩa thuật toán mà chúng ta đưa ra chưa phải là chặt chẽ, nó vẫn quá “cứng nhắc” trong những ứng dụng thực tế! Bởi vậy, chúng ta còn cần đến các thuật toán “xác suất”, tức là các thuật toán phụ thuộc vào một hay nhiều tham số ngẫu nhiên.

Những “thuật toán” này, về nguyên tắc không được gọi là thuật toán, vì có thể thuật toán không bao giờ kết thúc với một xác suất rất bé. Tuy nhiên, thực nghiệm chỉ ra rằng, các thuật toán xác suất thường hữu hiệu hơn các thuật toán không xác suất. Thậm chí, trong rất nhiều trường hợp, chỉ có các thuật toán xác suất mới sử dụng được.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Khi làm việc với các thuật toán xác suất, ta thường hay phải sử dụng các số “ngẫu nhiên”. Khái niệm chọn số ngẫu nhiên cũng cần được chính xác hoá. Thông thường người ta chế tạo một công cụ (môđun) để chọn số ngẫu nhiên nào đó.

Đối với các thuật toán xác suất, không thể nói đến thời gian tuyệt đối, mà chỉ có thể nói đến thời gian hy vọng (expected).

Để hình dung được phần nào “độ phức tạp” của các thuật toán khi làm việc với những số lớn, ta xem bảng dưới đây cho khoảng thời gian cần thiết để phân tích một số nguyên n ra thừa số bằng thuật toán nhanh nhất được biết hiện nay (ta xem máy tính sử dụng vào việc này có tốc độ 1 triệu phép tính trong 1 giây)

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Số chữ số thập phân	Số phép tính bit	Thời gian	
50	$1,4 \cdot 10^{10}$	3,9 giờ	
75	$9,0 \cdot 10^{12}$	104 ngày	
100	$2,3 \cdot 10^{15}$	74 năm	
200	$1,2 \cdot 10^{23}$	$3,8 \cdot 10^9$ năm	
300	$1,5 \cdot 10^{29}$	$4,9 \cdot 10^{15}$	năm
500	$1,3 \cdot 10^{39}$	$4,2 \cdot 10^{25}$	năm

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

§2. Thuật toán bình phương và nhân.

2.1 Định nghĩa. Thuật toán bình phương và nhân là thuật toán tính nhanh lũy thừa tự nhiên của một số nguyên, trong trường hợp cơ số là số nguyên có thể được rút gọn theo một môđun nào đó.

Phép nâng lên lũy thừa tự nhiên bậc n của số x (x được gọi là cơ số) được định nghĩa từ hệ thức $x^n = x.x \dots .x$ (n lần).

2.2 Thí dụ. Với $n = 35$, thông thường quá trình tính x^{35} thực hiện qua 34 bước nhân.

$$x \Rightarrow x^2 = x.x \Rightarrow x^3 = x^2.x \Rightarrow \dots \Rightarrow x^{35} = x^{34}.x$$

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

ta có thể giảm bớt các phép nhân bằng dãy phép tính sau:

$$x \Rightarrow x^2 \Rightarrow x^4 = (x^2)^2 \Rightarrow x^8 = (x^4)^2 \Rightarrow x^{16} = (x^8)^2$$

$$x^{17} = x^{16} \cdot x \Rightarrow x^{34} = (x^{17})^2 \Rightarrow x^{35} = x^{34} \cdot x.$$

với cách tính này ta chỉ thực hiện 6 phép nhân.

2.3 Thuật toán đệ quy.

2.3.1. Công thức đệ quy. Quá trình tính toán trên chính là quá trình tính nhờ công thức đệ quy:

Với $n = 0$ thì $x^n = 1$. Với $n > 0$ ta có công thức

$$f(n) = \begin{cases} (x^k)^2, & \text{khi } n = 2k \\ (x^k)^2 \cdot x, & \text{khi } n = 2k + 1 \end{cases}$$

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

2.3.2. Giải thuật đệ quy.

Giải thuật sau tính đệ quy $x^n \pmod m$.

```
def Power_Modulo(x,n,m):  
    if n==0: return 1  
    p=Power_Modulo(x,n//2, m)  
    if n%2==0: return (p*p)%m  
    else: return (p*p*x)%m
```

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

2.4. Thuật toán không đệ quy.

Trong giải thuật đệ quy trên đây ta xét tính chẵn lẻ của n và liên tục chia n cho 2 lấy phần nguyên cho đến khi $n = 0$. Thực chất quá trình này chính là tìm các bit của n . Do đó ta có thể thực hiện phép đổi ra số nhị phân trước sau đó tính lũy thừa theo quy tắc bình phương và nhân.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

- 2.4. Thuật toán không đệ quy.

Đổi n ra số nhị phân ghi vào mảng $b[1...k]$

```
def Base2(n):  
    b=[]  
    while n>0:  
        t=n%2  
        b.append(t)  
        n=n//2  
    b.reverse()  
    return b
```

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

- 2.4. Thuật toán không đệ quy.

Giải thuật không đệ quy

```
def Power_Modulo(x,n,m):  
    b = Base2(n)  
    p = 1  
    for i in range(len(b)):  
        p = (p*p)%m  
        if b[i]==1: p = (p*x)%m  
    return p
```


CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

2.5. Thí dụ. Tính $37^{27} \pmod{101}$. Đổi $n = 27 = (11011)_2$.

Bảng sau đây tính toán từng bước theo giá trị của các bit của 27.

Khởi tạo $p = 1$.

$b[i]$	$p = p^2$	$p = p \pmod{101}$	$p * x$	$p = \pmod{101}$
1	$1^2 = 1$	1	$1 * 37 = 37$	37
1	$37^2 = 1369$	56	$56 * 37 = 2072$	52
0	$52^2 = 2704$	78	-	78
1	$78^2 = 6084$	24	$24 * 37 = 888$	80
1	$80^2 = 6400$	37	$37 * 37 = 1369$	56

Như vậy ta có $37^{27} \pmod{101} = 56$

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

§3. Thuật toán Karatsuba nhân nhanh hai số nguyên lớn.

3.1 Thuật toán. Karatsuba là thuật toán nhân số lớn dựa trên tư tưởng chia để trị, có độ phức tạp $O(n^{\log_2 3})$, với n là số chữ số.

Công thức

Giả sử 2 số cần nhân là x và y trong hệ cơ số B . Đầu tiên ta chia mỗi số x và y thành 2 số nhỏ hơn như sau:

$$x = x_1 \times B^m + x_0; y = y_1 \times B^m + y_0$$

Với $x_0, y_0 < B^m, x_0 < x_1, y_0 < y_1$.

Khi đó $x \times y$ được tính bởi công thức:

$$xy = (B^{2m} + B^m)x_1y_1 + (B^m + 1)x_0y_0 - B^m(x_1 - x_0)(y_1 - y_0)$$

Việc tính $x \times y$ được đưa về 3 phép nhân các số có độ dài nhỏ hơn là x_0y_0, x_1y_1 và $(x_1 - x_0)(y_1 - y_0)$. Các phép nhân với B^m được thực hiện đơn giản bằng cách dịch chuyển các chữ số.

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

Thuật toán Karatsuba hiệu quả nhất khi ta chọn m gần với $n/2$ và khi số chữ số của x và y nhỏ hơn một giá trị nhất định

3.2. Cài đặt. Nhân 2 số nguyên dương mỗi số có độ dài tối đa là 300000. Cơ sở $B = 10^8$.

Với a, b lần lượt là độ dài của x, y . Sau đó ta tìm x_0, x_1, y_0, y_1 đơn giản bằng cách chia mỗi mảng x, y thành 2 mảng con, x_0 là m phần tử đầu tiên của x , x_1 là các phần tử còn lại.

Với cách tính này, ta đảm bảo $x_0 < x_1$ và $y_0 < y_1$, đồng thời m cũng gần với $n/2$ nhất có thể.

Chương trình trong Python3 như sau:

CHƯƠNG 3: TÍNH TRÊN SỐ NGUYÊN LỚN

```
def karat(x,y):
```

```
    if len(str(x)) == 1 or len(str(y)) == 1: return x*y
```

```
    else:
```

```
        m = max(len(str(x)),len(str(y)))
```

```
        m2 = m // 2
```

```
        a = x // 10**(m2)
```

```
        b = x % 10**(m2)
```

```
        c = y // 10**(m2)
```

```
        d = y % 10**(m2)
```

```
        z0 = karat(b,d)
```

```
        z1 = karat((a+b),(c+d))
```

```
        z2 = karat(a,c)
```

```
    return (z2 * 10**(2*m2)) + ((z1 - z2 - z0) * 10**(m2)) + (z0)
```